COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structure and Algorithm	**SECTION:**	**CPE21S2**
DATA PERFORMED:	**10 / 16 / 26**	**DATE SUBMITTED:**	10 / 20 / 26
NAME:	Bandigan, Christian C. , Nieles Cenndy M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _13p1_ Parallel Algorithm and Multithreading

1. PROCEDURE OUTPUT

Screenshot

HOA13 > procedure1.cpp > ...

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str){
  std::cout << "Printing integer: " << n << std::endl;
  std::cout << "Printing string: " << str << std::endl;
}

int main(){

  std::thread t1(print,10,"T.I.P.");
  t1.join();

  return 0;
}
```

Output

```
PS C:\Users\TIPQC\Documents\dsa\HOA13> c
.\procedure1 }
Printing integer: 10
Printing string: T.I.P.
```

Analysis

This program demonstrates the implementation of the fundamentals of multithreading using the thread and iostream and standard libraries. The terminal output confirms this orderly execution, displaying the sequential printed results of both the integer and the string as passed from the thread initialization, proving that the concurrent execution path was established and cleaned up correctly.

Table 13-1. Simple One-Threaded Example

Screenshot

HOA13 > ILOB > procedure2.cpp > ...

```cpp
#include <iostream>
#include <thread>
#include <string>
#include <vector>

void print(int n, const std::string &str){
  std::string msg = std::to_string(n) + " : " + str;
  std::cout << msg << std::endl;
}

int main(){
  std::vector<std::string> s = {
    "T.I.P.",
    "Competent",
    "Computer",
    "Engineers",
  };
  std::vector<std::thread> threads;

  for(int i = 0; i < s.size(); i++){
    threads.push_back(std::thread(print,i,s[i]));
  }

  for(auto &th : threads){
    th.join();
  }

  return 0;
}
```

Output

Bandigan

```
PS C:\Users\TIPQC\Documents\dsa\HOA13\ILOB> cd
if ($?) { .\procedure2 }
0 : T.I.P.3 : Engineers

2 : Computer
1 : Competent
```

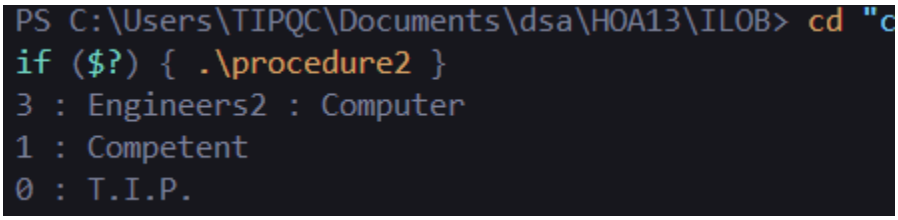
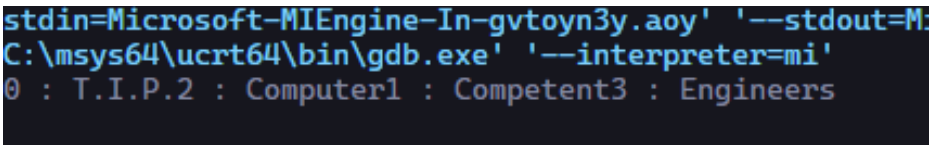
	 Nieles 
Analysis	This program uses the thread library to implement multithreading on printing a vector of strings. It simultaneously works on printing each string so the output depends on the backend on which string will be printed first.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as a reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.

Bandigan

-Multithreading allows a program to run multiple tasks at the same time. Advantages of this include better performance, better resource utilization, etc. But it also comes with its disadvantages, such as data race issues, harder debugging, etc.

Nieles

- Multithreading a program allows a single program to split itself into multiple concurrent execution paths that can run the output multiple times at the same time.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Bandigan

-Multithreading can be used to handle background tasks and make responsive UI and audio/video processing.

Nieles

- Multithreading can use it because it can prevent a program from locking up during the slow tasks and run its independent operations simultaneously.

3. Differentiate between parallelism and concurrency.

Bandigan

-Parallelism has copies of the same program running simultaneously to handle tasks on different data while

concurrency happens when tasks overlap.

Nieles
-The parallelism is doing a lot of things, while the concurrency is dealing with a lot of things at once.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

HOA13 > partB > main.cpp > add(int)

```cpp
#include <iostream>
#include <thread>

int globalVar;

void add(int n);

int main(){

    std::thread th1(add,10);
    std::thread th2(add,6);
    std::thread th3(add,7);

    std::cout << globalVar << std::endl;
    th1.join();
    std::cout << globalVar << std::endl;
    th2.join();
    std::cout << globalVar << std::endl;
    th3.join();

    return 0;
}

void add(int n){
    globalVar += n;
    return;
}
```

Output

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA13\partC> cd "c:\Users\User\OneDrive\
0
23
23
```

Analysis
Bandigan
- Based on the output of the multithreading exercise, the program first prints the global variable, which is equal to zero, and by the time th1.join() is called, all functions are already done and print the final output, which is 23. The output is unpredictable and based on the operating system, and the program can have data race issues since it's updating a global variable concurrently.

Nieles

-The output shows the challenges of multithreading programming, such as the conditions when multiple threads access and modify the variable concurrently. Initializing at zero and later showing the 23 shows how thread execution works.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

HOA13 > partC > mainC.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int size = 100;

int main(){
    int arr[size];
    int arr2[size];

    srand(time(0));
    for(int i = 0; i < size; i++){
    arr[i] = rand()%100;
    arr2[i] = arr[i];
    }

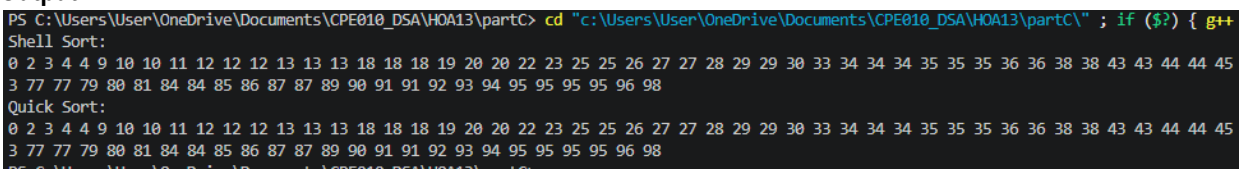
    std::thread th1(shellSort<int>,arr,size);
    std::thread th2(quickSort<int>,arr2,0,size-1);

    th1.join();
    th2.join();

    std::cout << "Shell Sort:"<<std::endl;
    for(int i = 0; i<size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
    std::cout << "Quick Sort:"<<std::endl;
    for(int i = 0; i<size; i++){
        std::cout << arr2[i]<< " ";
    }

    return 0;
}
```

Output



Analysis

Bandigan, and how to

- The output shows that the two sorting algorithms worked perfectly and efficiently by using multithreading.

Nieles

- The results of output show in the output both multithreading implementations of shell sort and quick sort to achieve correct ascending order sorting. It also shows the concurrent allows the different sorting algorithms to process data simultaneously and produce accurate, sorted results

Conclusion

Bandigan

- In this activity I learned the concept of concurrency and how this skill is essential, especially in our time where computers can have multiple cores. Specifically, we implemented multithreading on multiple programs using the thread library. Utilizing multithreading makes our programs have faster execution, better CPU utilization, improved performance, etc. Moreover, in the supplementary activity, we were able to answer questions about parallelism, concurrency, and multithreading to better understand all these concepts. We were also able to implement multithreading as a solution to a problem and on our past algorithms developed in the course.

Nieles

-This activity provided hands-on activity experience with multithreading and the importance of computing. Implementing thread programs demonstrates how to manage multiple execution paths and how effectively to manage the path and can radically enhance the utilization and overall program speed. And exploring these concepts through our supplementary exercises and past algorithms helped to bridge the gap between theoretical parallelism and practical implementation

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.